



“UIC: UNIVERSITY INSTITUTE OF COMPUTING”

Project Report

SUBMITTED BY:-

Student Name :- Katyan Gupta

BRANCH :- BCA(DATA SCIENCE)

UID :- 24BCD10063

SEMESTER :- 4th

SUBMITTED TO:-

Teacher Name :-Ms. Sushma (Asst.Prof.)

SUB CODE :- 24CAP-264

SECTION :- 24BCD-3(A)

SUB :- Advance web development

Healthcare System Web Application (MERN Stack)

1. Introduction

In today's digital era, healthcare systems are rapidly evolving to provide better accessibility, efficiency, and user experience. Traditional appointment booking systems are often manual, timeconsuming, and inefficient. Patients usually face challenges such as long waiting times, lack of realtime information, and difficulty in managing appointments.

To address these challenges, this project titled "**Healthcare System Web Application**" has been developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system enables users (patients and doctors) to interact through a digital platform, allowing seamless appointment booking, management, and tracking.

The primary goal of this project is to provide a **user-friendly, scalable, and efficient healthcare management system** that simplifies the process of connecting patients with healthcare professionals.

2. Objectives

The main objectives of this project are:

- To develop a **full-stack web application** using modern technologies.
- To allow patients to **register, login, and book appointments** with doctors.
- To enable doctors to **view appointments** and manage their schedules.
- To implement **secure authentication and authorization** using JWT.
- To design a **clean and responsive UI** for better user experience.
- To deploy the application on cloud platforms for real-world accessibility.
- To ensure **data storage and retrieval using MongoDB Atlas**.

3. Technology Stack

This project uses the MERN stack along with additional tools:

Frontend:

- React.js
- Tailwind CSS
- Axios (API communication)

- React Router DOM

Backend:

- Node.js
- Express.js
- JWT (JSON Web Tokens) for authentication
- bcrypt.js for password hashing
- MongoDB Atlas (Cloud Database)
- Mongoose ODM

Deployment:

- Frontend: Vercel
- Backend: Render
- Database: MongoDB Atlas

4. System Architecture

The application follows a **client-server architecture**:

- The **frontend (React)** interacts with users.
- The **backend (Node + Express)** handles logic and APIs.
- The **database (MongoDB)** stores user, doctor, and appointment data.

Flow:

1. User sends request from frontend
2. API call goes to backend
3. Backend processes request
4. Data is stored/retrieved from MongoDB
5. Response sent back to frontend

5. Features of the System

5.1 User Authentication

- Users can register and login securely.
- Passwords are hashed using bcrypt.
- JWT tokens are used for session management.

5.2 Role-Based Access

- Two main roles:
 - Patient
 - Doctor
- Different functionalities based on roles.

5.3 Doctor Listing

- Patients can view available doctors.
- Doctor details include name and specialization.

5.4 Appointment Booking

- Patients can book appointments with doctors.
- Date and time are recorded.
- Backend validates request before storing.

5.5 Appointment Management

- Users can view their appointments.
- Status tracking (pending, confirmed, etc.)

5.6 Dashboard

- Displays:
 - Total doctors
 - Appointments
 - Health status

6. Database Design

Collections Used:

User Collection

- `_id`
- `name` □ `email`
- `password` □ `role`

Doctor Collection

- `_id`
- `userId` (reference to User)
- `specialization`

Appointment Collection

- `_id`
- `patientId`

- doctorId
- date & time
- status

7. Implementation Details

7.1 Backend

- RESTful APIs were created using Express.js.
- Controllers handle business logic.
- Middleware used for authentication.
- Mongoose models define schema.

7.2 Frontend

- React components structured into:
 - Pages
 - Components
 - Services
 - Axios
- used for API calls.
- Tailwind CSS used for styling.

7.3 API Integration

- Base URL configured for deployed backend.
- Proper error handling implemented.
- Debugging done using console logs.

8. Challenges Faced

During development and deployment, several challenges were encountered:

8.1 CORS Issues

- Problem: Frontend couldn't communicate with backend.
- Solution: Configured CORS properly in backend.

8.2 MongoDB Connection Error

Problem: Atlas blocked Render server.

- Solution: Allowed access using 0.0.0.0/0.

8.3 API Route Errors (404)

- Problem: Incorrect endpoint usage.
- Solution: Added /api prefix in frontend.

8.4 Internal Server Errors (500)

- Problem: Missing environment variables.
- Solution: Configured .env in Render.

8.5 Bad Request (400)

- Problem: Wrong doctorId passed.
- Solution: Used correct doc._id.

9. Deployment

The application was successfully deployed:

Frontend (Vercel)

- Connected GitHub repository
- Build command: npm run build
- Output directory: dist

Backend (Render)

- Web service created
- Environment variables configured
- Auto-deploy enabled

Database (MongoDB Atlas)

- Cloud-hosted database
- Network access configured

10. Testing

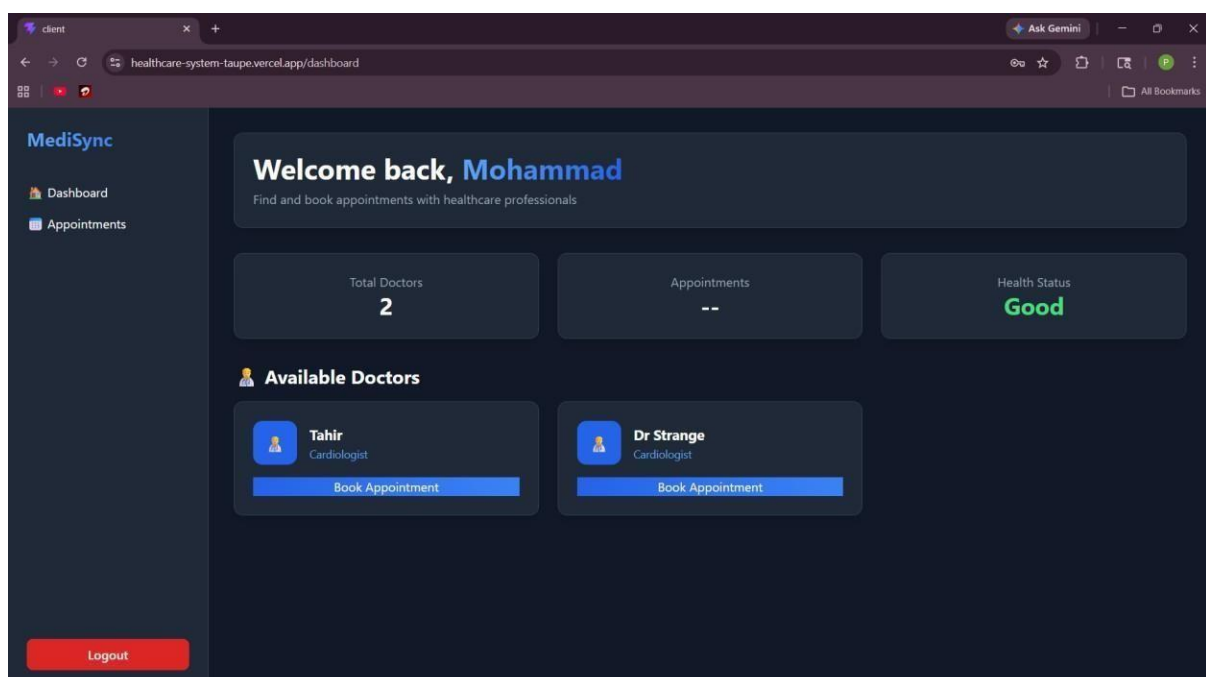
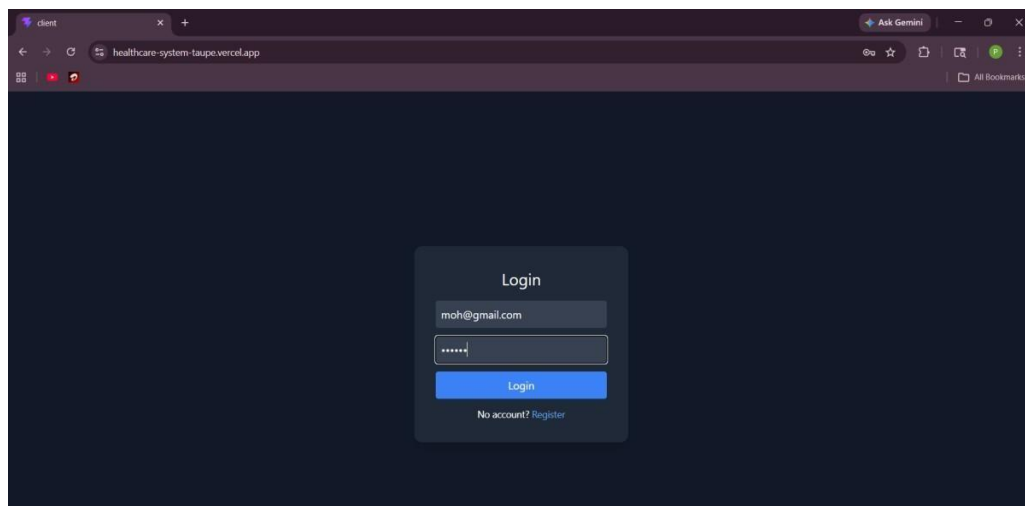
The application was tested for:

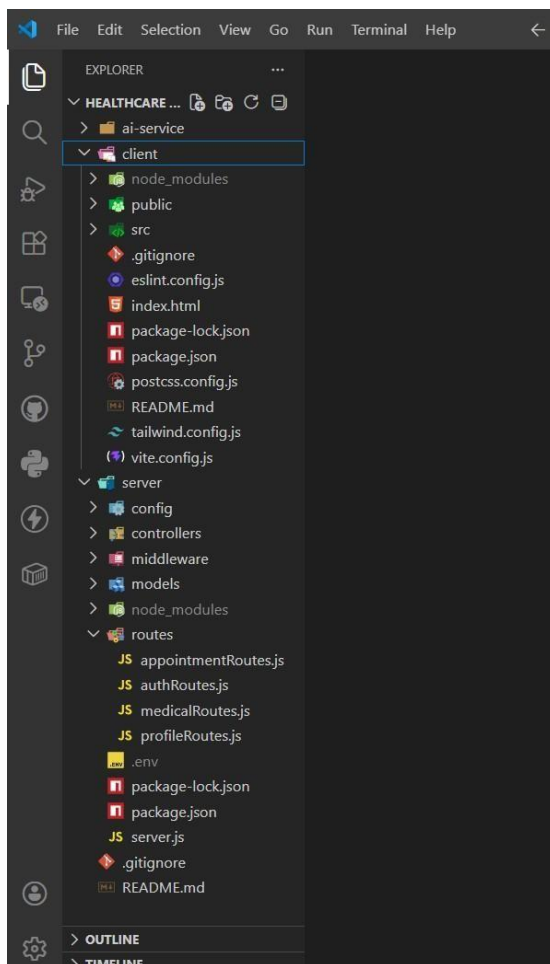
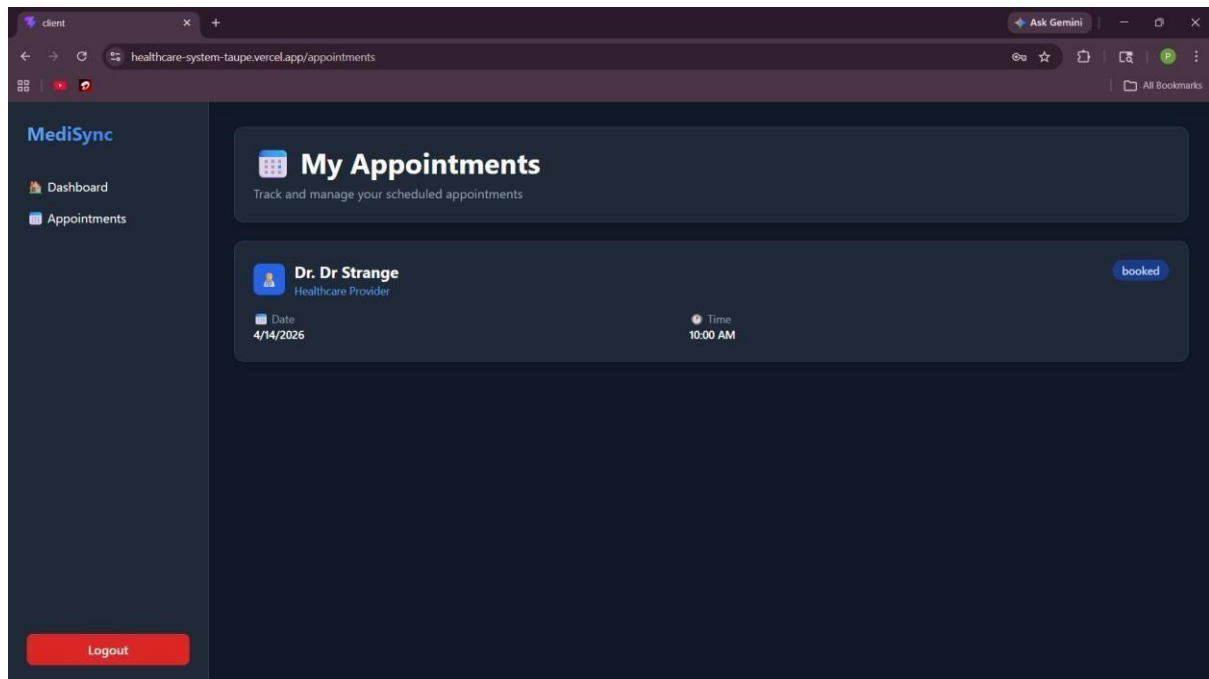
- User registration and login
- Doctor listing
- Appointment booking
- Appointment viewing
- API error handling

All functionalities worked successfully after debugging.

11. Advantages of the System

- User-friendly interface
- Secure authentication
- Scalable architecture
- Real-time interaction
- Cloud-based deployment
- Easy to extend with new features





12. Limitations

- No real-time notifications
- Limited role-based features
- No payment integration

- Basic appointment scheduling

13. Future Enhancements

- Add email/SMS notifications
- Implement video consultation
- Add payment gateway
- Improve doctor dashboard
- Add admin panel
- Use AI for health suggestions

14. Conclusion

The Healthcare System Web Application successfully demonstrates the implementation of a full-stack web application using the MERN stack. It provides a practical solution for managing healthcare appointments digitally.

The project helped in understanding:

- Full-stack development
- API integration
- Authentication mechanisms
- Deployment strategies
- Debugging real-world issues

This system can be further enhanced and scaled into a production-level healthcare platform.

GITHUB LINK: